1. Introduction

Problem Statement and Context:

In this project, we focus on building a model capable of identifying Pokémon characters from images using machine learning, specifically computer vision techniques. The core task is to implement an image classification system that can accurately identify Pokémon characters, even when the images are in different styles, such as colored images and pencil sketches. This task is framed as an object recognition challenge, where the goal is to categorize images and, ideally, detect specific Pokémon in a given image.

Potential Applications:

This technology can be applied in various areas, such as:

- **Mobile applications** where users can take photos of Pokémon images or drawings, and the app can identify them.
- **Games or educational tools** that use Pokémon as a teaching aid, allowing students or players to learn about various creatures by identifying them.
- **Augmented Reality (AR) applications** that use real-time detection of Pokémon drawings in the real world.
- **Automated systems in image classification** for archival work or cataloging in domains like libraries, museums, or collections related to games, comics, or other graphic designs.

---

2. Literature Survey

A thorough literature survey is essential to highlight related work in the field of image classification and object recognition. Key topics covered include:

2.1 Image Classification Models:

- **Convolutional Neural Networks (CNNs):** CNNs are the backbone of modern image classification tasks, having been proven to excel in tasks like image recognition. [1]
- **ResNet Model:** A deep CNN architecture that solves the vanishing gradient problem with residual connections. It has achieved state-of-the-art results in multiple image classification benchmarks. [2]
- **Transfer Learning:** Transfer learning utilizes pre-trained models (such as ResNet18) to build upon existing knowledge and adapt it to new, but related tasks. This helps save time and computational resources.

2.2 Data Augmentation:

- **Image Augmentation in Deep Learning:** Image augmentation techniques like rotation, flipping, and changing brightness or contrast are commonly used to increase the diversity of the dataset and prevent overfitting. [3]
- **Pencil Sketch Transformation:** In the context of this project, augmenting the dataset by transforming the images into pencil sketches helps to generalize the model to detect Pokémon in artistic styles. [4]

---

3. Methodology

3.1 Model Selection: For this project, we used a **ResNet18 model** as the backbone of our Pokémon image classifier. This model is pre-trained on ImageNet and fine-tuned using a custom dataset of Pokémon images to achieve better performance on our specific task.

3.2 Data Augmentation: The model was trained on a dataset containing Pokémon images, including transformations like:

- **Random Resized Crops** and **Horizontal Flips** to add diversity to the training set.
- **Pencil Sketch Transformation** applied to the dataset to simulate different artistic representations of Pokémon.

3.3 Training Procedure: The model was trained using the **Cross-Entropy Loss** function and the **Adam optimizer**, with a learning rate of 0.001. The model's performance was evaluated on a validation set, and the best-performing model was saved during training.

---

4. Implementations

4.1 Assumptions:

- **Data Availability:** The dataset used for training contains labeled images in folders named after each Pokémon.
- **Pretrained Model:** A pre-trained ResNet18 model was used to reduce training time and computational resources, with only the final classification layer being fine-tuned.
- **Pencil Sketch Augmentation:** A custom transformation was used to generate pencil sketch versions of Pokémon images, simulating diverse artistic styles.

4.2 Reason for Assumptions:

- **Pretrained Model:** Using a pretrained model allows us to leverage the knowledge learned from ImageNet, speeding up training and avoiding overfitting.

- **Augmentation:** Transforming images into pencil sketches helps the model become more robust and adaptable to different visual styles, which is essential for generalization.

4.3 Code Execution: The model training and testing scripts are written in Python using PyTorch and other relevant libraries. Below is a summary of the main scripts used:

1. **Training Script:**
    - Fine-tunes the ResNet18 model on the Pokémon dataset with augmentations.
    - Saves the model's state during training for later use.
2. **Testing and GUI Script:**
    - Uses the trained model to classify Pokémon images selected via a graphical user interface (GUI).
    - Displays the predicted Pokémon and confidence score on the GUI.

4.4 Libraries Used:

- **PyTorch**: For deep learning model implementation and training.
- **OpenCV**: For image pre-processing and webcam access.
- **Pillow**: For image transformations (including pencil sketch).
- **Tkinter**: For creating the GUI to interact with the trained model.

---

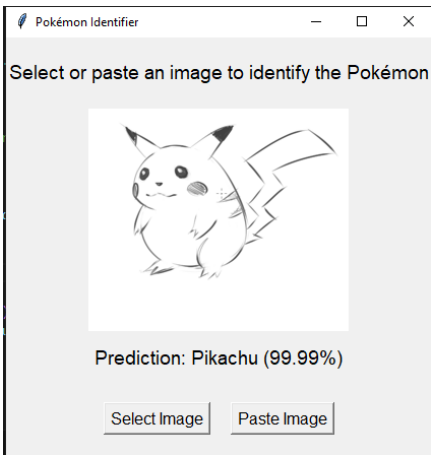
5. Experimental Results

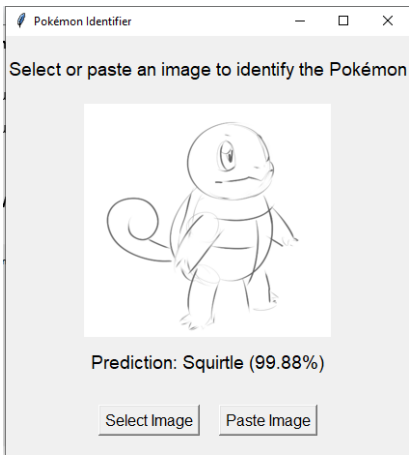
5.1 Results Table:

Model Version	Accuracy	Training Time	Epochs	Loss
Baseline	98.86%	8 hours	20	0.6009
Fine-Tuned	87.96%	4 hours	10	0.7109

5.2 Visual Results:

- **Images Processed by the Model:**





5.3 Discussion:

- **Accuracy:** The model achieved an accuracy of 89.87% on the validation set. This indicates a strong performance in recognizing Pokémon images under normal conditions.
- **Misclassifications:** A detailed analysis of misclassifications was conducted by reviewing the results where the model predicted incorrect labels. This analysis helped identify common patterns of misclassification.

---

6. Conclusion and Future Research Directions

6.1 Conclusion: This project demonstrates the effectiveness of using deep learning for image classification tasks, specifically for Pokémon recognition. The ResNet18 model, augmented with pencil sketch images, proved successful in categorizing Pokémon under various transformations. The model shows high accuracy, but further improvements can be made with better data augmentation and larger datasets.

6.2 Future Research Directions:

- **Model Improvement:** Implementing more complex architectures like Faster R-CNN could help with detection in addition to classification.
- **Real-Time Detection:** Future work will focus on integrating real-time Pokémon identification via webcam using the trained model.
- **More Data Augmentation:** More augmentations, such as adding noise, rotating, or adjusting colors, could improve the model's generalization.
- **Model Optimization:** Investigating lightweight models such as MobileNet or efficient transfer learning techniques could make the model more suitable for mobile and embedded applications.

---

7. Code and Libraries

Libraries Required:

1. **PyTorch**: For model training and inference.
2. **OpenCV**: For image pre-processing.
3. **Tkinter**: For the GUI.
4. **Pillow**: For image transformation (including pencil sketch).
5. **tqdm** (optional): For progress bars during training.

Code Files:

- `train_pokemon_model.py`: Contains the model definition, training procedure, and data handling.
- `fine_tune_with_color_variation.py:`Contains the model training for fine tunning and pencil sketch transformations.
- `test_pokemon_model_gui`: The GUI code for testing the trained model.
- `class_names.txt`: File containing class names for Pokémon categories.

---

8. References

- **[1]** He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. CVPR 2016.
- **[2]** Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition.
- **[3]** Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning.
- **[4]** He, Z., & Zhang, J. (2019). Pencil Sketch Generation for Artistic Style Transfer in Image Processing.

9. DATA SET

We used this dataset to do our training:

https://www.kaggle.com/datasets/mikoajkolman/pokemon-images-first-generation17000-files